

☐ Task 01: The function takes an array as input and recursively divides the array ~~until~~ until the length of the array is 1. Then merges the sorted halves using the merge function. This function takes 2 arrays as input and creates an empty list and appends the elements of the two array by comparing the two element from each array and taking the smaller one and then it finally returns the merged array.

- Divide and conquer approach, Time complexity $O(n \log n)$

☐ Task 02: The function recursively divides the array into ~~two~~ two parts until the length of the array is less than two the array then returns the element in the array in the base case. and compares two elements returned to "left" and "right". and returns the larger number.

- Divide and conquer approach, T.C. = $O(\log n)$

Task 3: The code uses a modified version of merge sort. First it recursively divides the array into individual elements. While merging the arrays a_1 and a_2 as a_1 is sorted when we check if $a_1[i] > a_2[j]$ if the statement is true then we add $\text{len}(a_2) - j$ to the count as if $a_1[i]$ is greater the rest elements on the right will be greater as well and then we return the count. ~~and recursively call the function to return the count to~~ Thus finding the total number of $a_1[i] > a_2[j]$ such that $i < j$.
 Divide and conquer $\rightarrow O(n \log n)$

Task 4:

The function takes an array and recursively divides the array until its length is 2 and returns $a[0] + a[1] * 2$ in the base case the sum is returned to l_m and r_m from the divided arrays. A function is called `cross_max` to check if any pair is present in between the two arrays by using two pointers one decrementing from the middle and one incrementing from the middle. Finding the max $a_1[i] + a_2[j]$ such that $i < j$ the function then returns the sum to cm . ~~the~~ The max among of l_m, r_m and cm is then returned.

Divide and conquer $\rightarrow O(n \log n)$

Task 5

while left < right the function calls a function get-pivot which works by picking a random index and swapping it with the right most index the code then sets the right most index as pivot and initializes two pointers i and j at 0 and pivot-1 indexes. The function then runs a while loop while i < j and an inner loop to increment i until its element is smaller larger than pivot element and decrements j until its lower than the pivot element. The function then swaps arr[i] and arr[j] if i < j. The function then swaps the pivot element with the i-th element at arr[i] ensuring the elements at the left of the pivot element is smaller and the element on the right is larger. the function then returns the pivot index. the function then recursively calls quick sort on the left and right elements of the pivot until the array is sorted.

Task 6: Implements quicksort until the k th pivot is at the $k-1$ th index if the get pivot function returns p which is equal to $k-1$ then the element is returned if the pivot is greater or less than $k-1$ then the function recursively calls the get pivot by incrementing the left pointer to $pivot+1$ or decrementing the right pointer to $pivot-1$ to shorten the problem size. as this occurs until the $k-1$ th element is found.